FORMAN CHRISTIAN COLLEGE

(A Chartered University)

COMP 451 — Compiler Construction

Course Code:	COMP 451
Session:	Spring 2026
Assignment:	Class Project
Title:	Lexical Analyzer & Operator Precedence Parser
Submitted By:	Syed Ali Shahrez (271047420) Zaema Faisal (261034399)
Submission Date:	May 24, 2026

1. Introduction

This project implements two fundamental components of a compiler front-end, written in C: a Lexical Analyzer (Task 1) and an Operator Precedence Parser with evaluation (Task 2).

Compilers transform human-readable source code into executable programs through several well-defined phases. The first phase is lexical analysis, which scans the raw input character-by-character and groups characters into meaningful units called tokens. The second phase is parsing, which checks that the stream of tokens forms a grammatically valid program and derives the structure needed for further processing.

In this project, the input is a simple arithmetic expression composed of lowercase single-letter identifiers (e.g. a, b, c) and the four binary arithmetic operators (+, -, *, /), terminated by a sentinel symbol $. Task 1 scans such an expression and classifies each character as either an identifier, an operator, or an invalid token. Task 2 takes that token stream, solicits numeric values for the identifiers from the user, and then applies an operator precedence parsing algorithm to evaluate the expression while displaying the full stack-state table at each step.

The project demonstrates core compiler construction concepts including tokenization, precedence-driven parsing, and expression evaluation using a dual-stack architecture.

2. Task 1: Lexical Analyzer

2.1 Overview

The lexical analyzer (lexer.c) accepts an arithmetic expression as a command-line argument, scans it left to right, and prints the token type for each character encountered. It stops at the sentinel $ and enforces a maximum of five operands.

2.2 Algorithm / Logic

- Validate that the expression is provided as a command-line argument and that it ends with the sentinel character $.
- Iterate through each character of the input string from left to right, stopping when $ is encountered.
- For each character, classify it into one of three categories: lowercase letter (identifier), arithmetic operator (+, -, *, /), or invalid token.
- If an identifier is found, increment the operand counter. If the count exceeds five, print an error and terminate.
- If an invalid token is found, print an error message and terminate immediately.
- Print the token type for each valid token as it is identified.

2.3 Code Description

Input Validation

The program first checks that argc >= 2, ensuring an expression was passed on the command line. It then verifies that the last character of the input string is $. If either check fails, the program exits with an appropriate error message.

```
if (argc < 2) { fprintf(stderr, "Usage: ..."); return 1; }
if (input[len - 1] != '$') { fprintf(stderr, "Error: ..."); return 1; }
```

Token Classification Loop

The core logic is a for-loop that iterates over each character. A chain of if-else conditions checks whether the character is $, a lowercase letter (islower()), a valid operator, or anything else. The islower() function from <ctype.h> conveniently identifies single lowercase letter identifiers.

```
for (int i = 0; i < len; i++) {
    char ch = input[i];
    if (ch == '$')
       break;
    else if (islower(ch))   { /* identifier */ operand_count++; }
    else if (ch == '+' || ch == '-' || ch == '*' || ch == '/')
                            { /* operator */ }
    else                    { /* invalid */ return 1; }
}
```

Operand Limit Enforcement

A counter operand_count is incremented each time a lowercase identifier is found. If it exceeds the defined maximum (MAX_OPERANDS = 5), the program prints an error and exits. This enforces the project specification's constraint of at most five operands.

2.4 Sample Outputs

Sample Run 1 — Valid expression

```
$ ./lexer a+b-c*d/a$
Program finds following tokens in the expression:
Expression received: a+b-c*d/a$
int literal found: a
Arithmetic operator: +
int literal found: b
Arithmetic operator: -
int literal found: c
Arithmetic operator: *
int literal found: d
Arithmetic operator: /
int literal found: a
```

Each token is correctly identified. The expression has five operands (a, b, c, d, a) and four operators, all well within limits.

Sample Run 2 — Two consecutive operators (valid)

```
$ ./lexer a+b*/a$
int literal found: a
Arithmetic operator: +
int literal found: b
Arithmetic operator: *
Arithmetic operator: /
int literal found: a
```

The lexer does not perform syntactic validation — consecutive operators are tokenized without error, since that is the responsibility of the parser.

Sample Run 3 — Invalid token

```
$ ./lexer a+b@c*a$
int literal found: a
Arithmetic operator: +
int literal found: b
Invalid token encountered. Program terminated prematurely.
```

When the invalid character @ is encountered, the lexer immediately prints an error and terminates, as required by the specification.

3. Task 2: Operator Precedence Parser

3.1 Overview

The parser (parser.c) accepts the same expression format on the command line. It first asks the user to supply integer values for each unique identifier in the expression. It then runs an operator precedence parsing algorithm, displaying the full stack-state table at every step, and finally prints the computed result.

The grammar used is:

E → E + E | E − E | E * E | E / E | id

This grammar is ambiguous and left-recursive, but operator precedence parsing sidesteps both issues by using a precedence relation table rather than a traditional automaton.

3.2 Precedence Relation Table

The table below encodes the precedence relationships between terminal symbols. A < (shift) means the stack top has lower precedence than the incoming token, so we push. A > (reduce) means the stack top has higher precedence, so we pop and evaluate. The table is stored in the code as T[NR][NR] with 1 = shift and 2 = reduce.

	Id	+	−	*	/	$
Id	>	>	>	>	>	
+	<	>	>	<	<	>
−	<	>	>	<	<	>
*	<	>	>	>	>	>
/	<	>	>	>	>	>
$	<	<	<	<	<	

Reading the table: row = top terminal on stack, column = current input token. For example, row + and column * gives < (shift), which is why multiplication binds tighter than addition.

3.3 Algorithm / Logic

- Push the sentinel $ onto the display stack and set the input pointer to the start of the expression.
- Each iteration: look at the current input token and the top terminal on the display stack. Look up T[top][current] to decide the action.
- SHIFT (T value = 1): push the current token onto the display stack; if it is an identifier also push its numeric value onto the value stack; advance the input pointer.
- REDUCE (T value = 2): if the top of the display stack is an identifier, pop it (its value is

already on the value stack). If it is an operator, pop the operator, pop two values from the value stack, compute the result, and push the result back.

- ACCEPT: when both the stack top and the current input are $, the expression has been fully parsed. Print 'Accepted' and report the value at the top of the value stack.
- ERROR: if the table entry is 0 and the accept condition is not met, print a syntax error message and exit.

3.4 User-Defined Functions and Data Structures

Display Stack (ds[])

A character array treated as a stack. Functions dpush(c), dpop(), and dpeek() implement the standard push, pop, and peek operations. This stack holds the token characters and drives all precedence decisions.

Value Stack (vs[])

A double array treated as a stack. Functions vpush(x), vpop_(), and vpeek() handle numeric values. Identifiers push their user-supplied value when shifted. Operators consume two values and push a result when reduced.

tidx(c)

Maps a character to its column/row index in the precedence table: lowercase letters → 0, + → 1, − → 2, * → 3, / → 4, $ → 5. Returns -1 for unrecognised characters.

topterm()

Scans the display stack from top to bottom and returns the tidx() of the first terminal symbol found. This is used to obtain the row index for the precedence table lookup. It defaults to 5 ($) if the stack is empty.

pds(w)

Prints the current contents of the display stack left-to-right (bottom to top), padded to width w. Used to format the stack column of the output table.

3.5 Code Description

Identifier Collection and Value Input

Before parsing begins, the program scans the expression for unique lowercase letters and asks the user for a numeric value for each. Values are stored in the array idv[26], indexed by letter (idv['a'-'a'] = value of a, etc.).

```
for (int i = 0; i < len-1; i++)
    if (islower(inp[i]) && !seen[inp[i]-'a'])
        { seen[inp[i]-'a']=1; ids[nc++]=inp[i]; }

for (int i = 0; i < nc; i++) {
    printf("Value of %c: ", ids[i]);
    scanf("%lf", &idv[ids[i]-'a']);
}
```

Main Parsing Loop

The loop reads the current input character, finds the top terminal via topterm(), and looks up the precedence table. The result drives either a shift or a reduce step.

```
for (;;) {
    char cur = inp[pos];
    int  ti  = topterm();   // top terminal index
    int  ci  = tidx(cur);   // current token index

    if (ti==5 && ci==5) { /* Accept */ break; }

    int rel = T[ti][ci];
    if (rel == 1) { /* SHIFT */ ... }
    else if (rel == 2) { /* REDUCE */ ... }
    else { /* Error */ return 1; }
}
```

Shift Action

When the table says shift, the current token is pushed onto the display stack. If it is an identifier, its numeric value (from idv[]) is also pushed onto the value stack. The input pointer advances to the next character.

```
dpush(cur);
if (islower(cur)) vpush(idv[cur-'a']);
pos++;
```

Reduce Action

When the table says reduce, the top of the display stack is examined. If it is an identifier, it is simply popped from the display stack (its value is already on the value stack from when it was shifted). If it is an operator, the operator is popped, two values are popped from the value stack, the operation is computed, and the result is pushed back.

```
char top = dpeek();
if (islower(top)) {
    dpop();   // value already on value stack
} else if (top=='+'||top=='-'||top=='*'||top=='/') {
    char op = top; dpop();
    double r = vpop_(), l = vpop_();
    double res = (op=='+')? l+r : (op=='-')? l-r :
                 (op=='*')? l*r : l/r;
    vpush(res);
}
```

3.6 Sample Output

For the expression a+b*c$ with a=2, b=5, c=10:

Stack	Input	Action
$	a+b*c$	
$a	+b*c$	Push
$	+b*c$	Pop
$+	b*c$	Push
$+b	*c$	Push
$+	*c$	Pop
$+*	c$	Push
$+*c	$	Push
$+*	$	Pop
$+	$	Pop
$	$	Pop
$	$	Accepted

```
The output of the given expression is: 52
```

The result 52 is correct: a + b*c = 2 + 5*10 = 2 + 50 = 52. The parser correctly gave * higher precedence than +.

3.7 Additional Functionalities and Exclusions

Additional Functionalities

- Division by zero is detected at runtime and an appropriate error message is printed before the program exits cleanly.
- The final result is printed as an integer if it has no fractional part, or as a floating-point value (up to 6 significant digits) otherwise. This allows the parser to handle both integer and non-integer division results correctly.
- Input validation rejects any character that is not a lowercase letter, an arithmetic operator, or the sentinel $, with a descriptive error message.

Exclusions

- Parentheses are not supported. The grammar and precedence table are designed for flat arithmetic expressions only.
- Multi-character identifiers and integer literals are not supported; only single lowercase letters are valid operands.
- Unary minus is not handled. The minus operator is treated strictly as a binary infix operator.

4. References

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Pearson Addison Wesley.
- COMP 451 Class Project Handout, Spring 2026, Forman Christian College.
- Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.